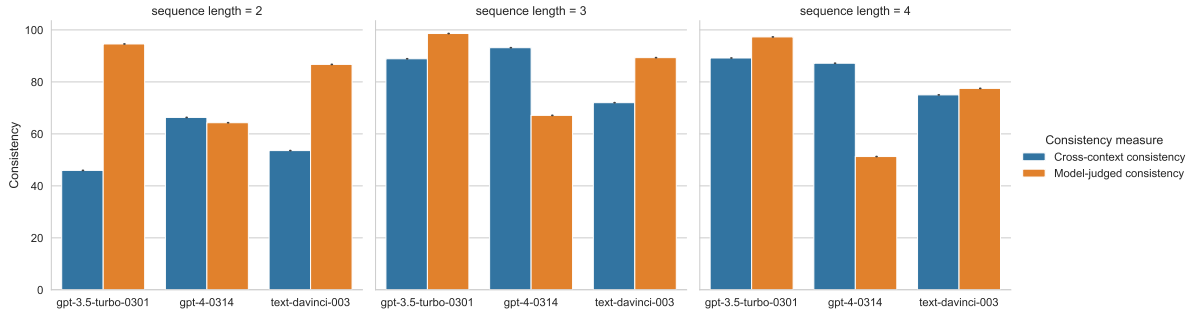

Figure 1: Cross-context consistency (orange). Model-judged consistency (blue); this drops drastically for `gpt-4`, which underestimates the consistency across answers itself produced.

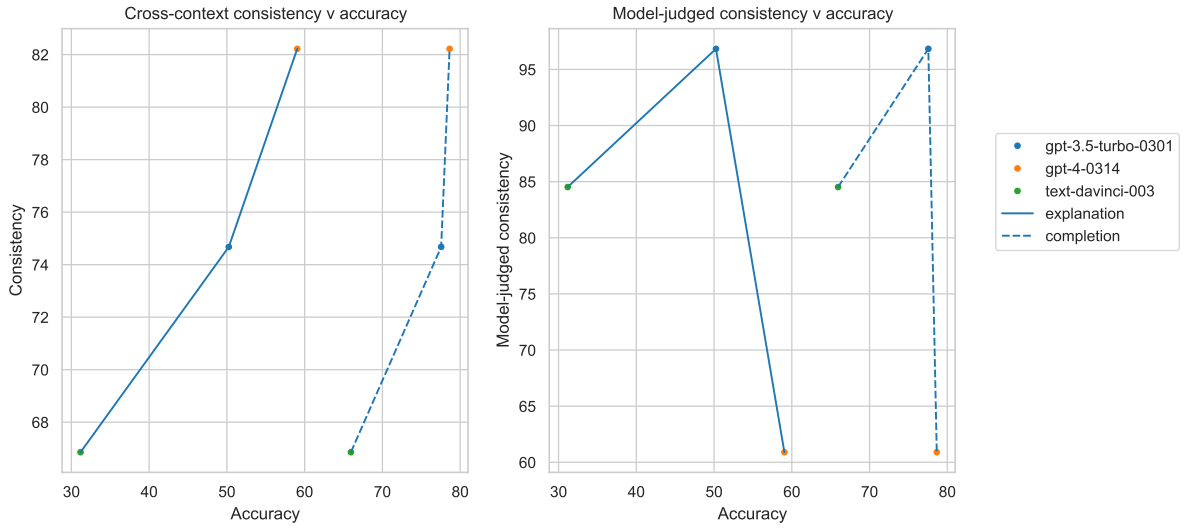

Figure 2: Explanation and sequence completion accuracies plotted against cross-context consistency and model-judged consistency (mean over sequence lengths). Further illustration of `gpt-4`'s inability to correctly assess its own consistency despite being much more consistent.

3.3 Consistency and Capability

Figure 2 presents the results from §3.1 and §3.2 plotted together. This analysis investigates the degree to which model capability relate separately with *cross-context consistency*, and *model-judged consistency*. We see as capability increases so does *cross-context consistency* but, the most capable model `gpt-4` is worse evaluating its own consistency.

Additionally, we compute expected consistency if correct completion-explanation pairs were chosen uniformly randomly at different capability thresholds. Table 3 illustrates cross-context consistency performance by our models and expected random consistency based on the average performance of each model on explanation and sequence completion accuracy. This tells us how consistent we should expect models to perform at different capability levels if they chose their completion responses independently from their explanations. Note that a model could score perfectly on the capability evaluations and consistency evaluations while having no self-consistency whatsoever. What we find is that models approach perfect consistency rapidly with capability increases.

Model	Average consistency (%) Actual	Random
`text-davinci-003`	66.86	8.50
`gpt-3.5-turbo`	74.68	10.02
`gpt-4`	82.22	15.22

Table 3: Average cross-context (Actual) consistency across settings in Figure 1 and consistency we'd expect to see (Random) if valid answers were selected uniform randomly given the average accuracy performance for each model.

[4]Given the nature of black-box API-based evaluation, it is possible greedy decoding doesn't ensure determinism (e.g. because of sparse mixture of experts routing considerations).

4 Robustness Checks for Consistency

We conducted further experiments to better understand how robust these results were to changes in experimental protocol by using a range of different prompts. We consider: (1) *speaker changes* in which we prompt the model as if the sequences were generated by different speakers; (2) *change of base* in which the sequence integers are presented in base 2 instead of base 10; and (2) *sequence length changes*. Full results are given in §C.

4.1 Consistency Across Speaker Changes

The first robustness experiment was designed to investigate the robustness of self-consistency of models when asked to simulate different speakers. This was intended to investigate whether models could be prompted to simulate more or less self-consistent speakers, which would determine whether models should be expected to be self-consistent by default or whether the previous results were artifacts of arbitrary features of the prompt.

To do this, we again conducted the same experiments as §3, now varying the initial instruction given to the model. These instructions were split into two separate components which we varied independently: what task we wanted the model to complete, and which speaker we wanted the model to simulate completing that task. We used three different task prompts, which one might expect to correspond to three different levels of consistency: the self-consistent prompts asked explicitly for a pair of responses which matched each other; the most likely prompts asked for the most likely continuation / explanation (most likely); and the random prompt asked the model to choose responses randomly when there was ambiguity about the correct answer. The prompts in full can be found in Appendix B. For example, the random explanation prompt was "Assume the sequence is generated by some deterministic function. If multiple functions could generate the sequence, choose the corresponding continuation randomly".

The first plot in Figure 3 shows representative results when varying the task prompt on correctness and consistency. If the models were capable of computing multiple continuations, and merely appeared self-consistent by dropping other possibilities, then we might expect there to be variable self-consistency, e.g., higher on the self-consistency prompt, and lowest on the random prompt. Empirically, we found that prompting the models with these different tasks had little influence on the proportion of answers that were self-consistent. This was found both for sequences of length 4 and 2. Even in the case where we were able to elicit a high proportion of correct answers being inconsistent using the most likely prompt, we do not see large changes in the number of inconsistent responses when varying the task prompt. This serves as strong evidence that the relationship between capability and consistency is unaffected by task prompt.

4.2 Consistency Across Base Changes

In this robustness experiment, we investigate what impact the base representation of functions and sequences had on capabilities and consistency of the models. This was intended to investigate the relationship between model capability and self-consistency while holding model type and training constant. We hypothesised that bases besides base 10 would be more difficult for the model. We again prompted the model to produce a continuation of a sequence and an explanation for the sequence, although the sequences were now in base 2, and the functions were expected to output base 2 representations of integers.

The second plot in Figure 3 presents a correlation analysis for this experiment, considering both base 10 and base 2 responses. It demonstrates a very strong correlation between the model generating correct explanations and being self-consistent, suggesting that this trend is robust across bases and, thus, task difficulty.

5 Distributional Analysis of Model Consistency

5.1 Models Do Not Converge to Calculating a Unique Solution

In the analysis so far, greedy sampling was used throughout. We now pose a follow-up question: Given models increasingly converge to self-consistency, preferring a unique answer, to what extent do these models calculate representations of other alternative answers? And, when models place high probability on alternative answers, can they verbalize these alternative solutions serially?

Specifically for models that were fine-tuned with RLHF (Christiano et al., 2017; Ouyang et al., 2022), the output probabilities may not be well-calibrated to the relative frequency of tokens if the objective

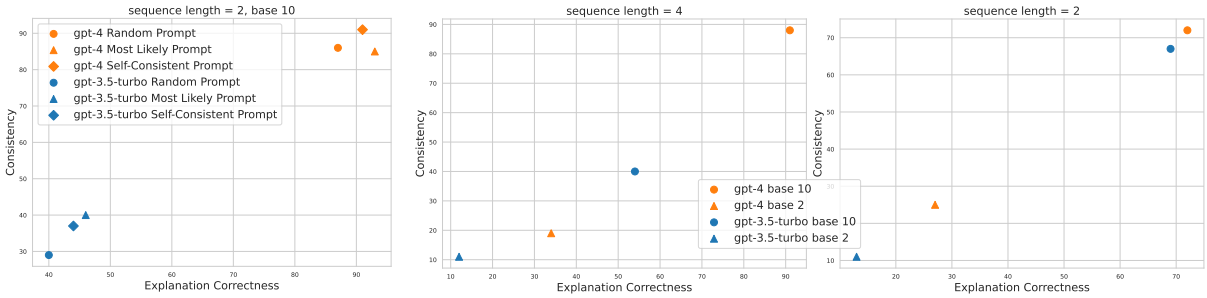

Figure 3: Cross-context consistency plotted against explanation correctness, varying either the role prompt (left-hand side) or the base-representation of the integer sequences being evaluated on (middle and right-hand side).

of RLHF encourages models to allocate probability mass narrowly (Kadavath et al., 2022). Hence, models' token probability distribution may not be reflective of their credences. While the models may be uncalibrated, we make a weaker assumption below that model output probabilities are *non-parametrically calibrated*: higher probability mass implies higher credence.

Applying this assumption to our setting, given initial ambiguous sequence, S_n, generating rules $\{F\}$, we can determine whether a model has calculated an alternative correct sequence completion, c', other than the modal greedy-decoded solution by verifying that:

$$P(c'|S_n) > P(z|S_n) \text{ for all } z \in \mathbb{N} \setminus C \quad (1)$$

where C is the set of correct continuations of S_n and $\mathbb{N}$ is the set of all continuations.

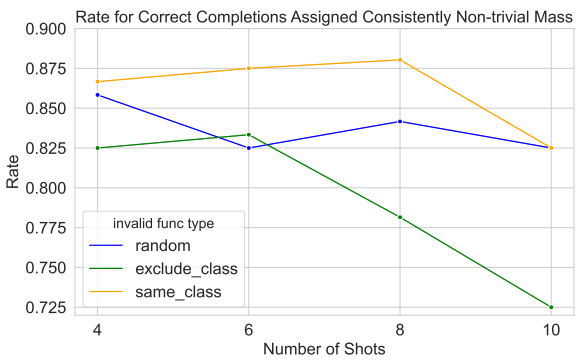

Figure 4: Rate at which correct completion alternatives are assigned non-trivial probability mass by function class sampled for few shot exemplars. Across sampling methods, that rate is relatively high indicating a consistent consideration of correct alternatives across contexts.

For input data, we use the full set of 40 functions that generated ambiguous sequences (see Algorithm 1). We prompt the model using the same prompts for integer sequence completion as in §3.2 and determine whether alternative correct answers rank higher than all incorrect answers. In the explanation case, we change the prompt to be a multiple-choice task so that only a single token is needed to evaluate the above inequality. Despite this simplification, the rate at which high probability mass is spread on alternatives is much lower, with the best rate of 0.3. This indicates that correct alternatives are not generally considered. This may be because the computation of correct alternative explanations is much more computationally intensive and more difficult than the computation of correct alternative sequence completions.

We use `text-davinci-003` for our experiments since it is the only model that has token log probabilities accessible from the public API.[5] Since the API returns up to $n_{logprobs} = 5$ probabilities for top output tokens, we assess if any incorrect answer was listed and whether the correct all rank higher. When a possible correct answer is not in the top output tokens but an incorrect one is, we consider the test failed. Finally, we control the sampling methods for few-shot example: `exclude_class` indicates that we exclude the sequence generating functions that are from the same class (See classes used here Table 4), `same_class` draws functions from the same function class and `random` draws those randomly across function classes. These controls are designed to give us insight on whether the class of functions used makes considering correct answers over incorrect ones more challenging. The evaluations are averaged over three runs.

Figure 4 illustrates that in the sequence completion case, `text-davinci-003` almost always assigns high probability to correct alternative an-

[5] https://platform.openai.com/docs/api-reference/completions/create